

# ngspice-46 vs. Spectre — Feature Gap Analysis

Ngspice + OpenVAF Enhancements project

July 2026

## Contents

ngspice-46 vs. a commercial simulator (Spectre) — feature gap analysis

1

## ngspice-46 vs. a commercial simulator (Spectre) — feature gap analysis

A capability comparison of the ngspice-46 build in this repository against a Spectre-class commercial simulator, to show where the open-source tool already matches the commercial one and where the gaps are. Grounded in a direct read of the ngspice-46/src/ tree (analyses, solver, devices, RF, convergence, parallelism), not marketing feature lists.

Legend: ✓ present / on par · ⚠ partial, experimental, or via a workaround · ✗ absent. "Spectre" stands in for the commercial reference (Spectre / SpectreRF / Spectre X / RelXpert feature set).

The one column that matters is ngspice — Spectre has essentially everything, so its column is a baseline of ✓. The point of the table is where ngspice's mark is ⚠ or ✗.

Context: the Verilog-A / OSDI device side is not a gap — thanks to the OpenVAF-reloaded compiler in this repository it is on par with (often ahead of) commercial Verilog-A support. The gaps below are all on the simulator/analysis side.

## Core numerics

| Category    | Feature                                         | ngspice        | Spectre |
|-------------|-------------------------------------------------|----------------|---------|
| Core solver | Sparse LU (KLU) direct solver                   | ✓ <sup>1</sup> | ✓       |
| Core solver | Legacy Sparse1.3 fallback                       | ✓              | ✓       |
| Core solver | Parallel / partitioned / GPU linear solve       | ✗              | ✓       |
| Core solver | Matrix reordering + scaling beyond KLU defaults | ⚠              | ✓       |
| Integration | Trapezoidal + variable-order Gear               | ✓              | ✓       |
| Integration | Advanced LTE-based step/order control           | ⚠              | ✓       |
| Convergence | gmin stepping + source stepping homotopy        | ✓              | ✓       |
| Convergence | Pseudo-transient / dynamic-gmin continuation    | ⚠              | ✓       |
| Convergence | Damped / trust-region (globalized) Newton       | ⚠              | ✓       |
| Convergence | Coordinated accuracy presets (errpreset)        | ✓              | ✓       |

The two convergence ⚠ rows: ngspice's DC path is not naked — it has per-device junction limiting (30 device families), an optional `nodedamping` step clamp, and a multi-stage homotopy cascade (`dynamic_gmin` → `new_gmin` → `spice3_gmin` → source stepping). What it lacks vs. commercial tools is a principled globalized Newton (residual-based trust-region / Armijo) and a classic pseudo-transient ( $\dot{X}$ -embedded) continuation. [Enhancement-111](#) adds the former: `.option linesearch`, an Armijo backtracking line search on a new KCL-residual merit  $\|F\|=\|G\cdot x-b\|$  (result-neutral, off by default) — see the [implementation write-up](#). [Enhancement-112](#) then extended it to the KLU solver — it originally ran only under Sparse 1.3 and segfaulted under `.option klu` — so the line search now works under both linear solvers, with a residual-merit sequence numerically identical between them.

<sup>1</sup> KLU is compiled in (SuiteSparse, statically linked) but is not the default in this build — the default direct solver is Sparse 1.3; KLU is selected with `.option klu`. The two agree on DC / AC / transient, and — since [Enhancement-113](#) — on noise and single-ended pole-zero as well (the KLU adjoint solve was doing a non-transposed solve, silently wrong

on asymmetric matrices; now fixed), and — since [Enhancement-114](#) — on DC/AC sensitivity (`.sens`), and — since [Enhancement-115](#) — on distortion (`.disto`) too. The only analysis still Sparse-only under KLU is balanced-output pole-zero; KLU is also less robust on stiff transient edges. Full behavior, defaults, and a solver-by-solver sweep of the example suite: [KLU vs. Sparse 1.3 solver notes](#).

## Standard analyses (analog)

| Category | Feature                                                                              | ngspice | Spectre |
|----------|--------------------------------------------------------------------------------------|---------|---------|
| Analyses | DC operating point / DC sweep                                                        | ✓       | ✓       |
| Analyses | Transient ( <code>.tran</code> )                                                     | ✓       | ✓       |
| Analyses | AC small-signal ( <code>.ac</code> )                                                 | ✓       | ✓       |
| Analyses | Noise, small-signal ( <code>.noise</code> , <code>onoise/inoise</code> + integrated) | ✓       | ✓       |
| Analyses | Pole-zero ( <code>.pz</code> )                                                       | ✓       | ✓       |
| Analyses | Transfer function ( <code>.tf</code> )                                               | ✓       | ✓       |
| Analyses | DC sensitivity ( <code>.sens</code> )                                                | ✓       | ✓       |
| Analyses | Distortion ( <code>.disto</code> )                                                   | ✓       | ✓       |
| Analyses | Measurement / post-processing ( <code>.meas</code> )                                 | ✓       | ✓       |

## RF / periodic steady-state suite

| Category | Feature                                                  | ngspice        | Spectre |
|----------|----------------------------------------------------------|----------------|---------|
| RF       | S-parameter analysis + noise figure ( <code>.sp</code> ) | ✓              | ✓       |
| RF       | Touchstone ( <code>.snp</code> ) import / export         | ✓              | ✓       |
| RF       | Periodic steady state (PSS)                              | ✓ <sup>1</sup> | ✓       |
| RF       | Harmonic Balance (HB)                                    | ✗              | ✓       |
| RF       | Periodic / phase noise (Pnoise)                          | △ <sup>3</sup> | ✓       |
| RF       | Periodic AC (PAC, conversion gain)                       | ✓ <sup>2</sup> | ✓       |
| RF       | Periodic transfer function (PXF)                         | ✓ <sup>4</sup> | ✓       |
| RF       | Periodic S-parameters (PSP)                              | ✗              | ✓       |
| RF       | Quasi-periodic / multi-tone (QPSS / QPAC)                | ✗              | ✓       |
| RF       | Envelope following                                       | ✗              | ✓       |

<sup>1</sup> PSS was △(experimental —enable-pss flag, so `.pss` was unimplemented in shipped builds, and ~230 lines of shooting-loop trace per run). Since [Enhancement-117](#) it is built by default, quiet (trace behind `set ngdebug`), and verified against the analytic AC response; [Enhancement-118](#) then made it run under both linear solvers (KLU had hung on a timestep explosion from reused refactor pivots — now a full re-factor is forced each PSS step under KLU). It is still a brute-force shooting method and remains the foundation for the periodic small-signal analyses below. HB exists only as a `WITH_HB` stub that returns “unsupported”.

<sup>2</sup> PAC is ✓ (complete periodic-AC analysis). The full chain is built and verified: [Enhancement-119](#) retains the periodic operating point, [Enhancement-120](#) extracts the periodic Jacobian harmonics  $G_k, C_k$ , [Enhancement-121](#) assembles and solves the  $(2M+1)N$  harmonic conversion matrix  $H_{\{nm\}} = G_{\{n-m\}} + j\omega_m C_{\{n-m\}}$ , [Enhancement-122](#) wraps it in a user-facing `.pac` command (runs PSS, sweeps the input frequency, writes a complex plot), and [Enhancement-123](#) finishes it with a netlist-referenced small-signal source stimulus (a true transfer / conversion gain) and multi-sideband output — every conversion sideband  $f_{in} + k \cdot f_0$  as its own named vector (`<node>_usb<k>/<node>_lsb<k>`). Verified on the RC: unit-current sideband-0 reproduces the AC driving-point impedance and, with `V1 AC 1`, sideband-0 reproduces the low-pass transfer  $1/\sqrt{1+(2\pi fRC)^2}$  across the band while the conversion sidebands sit at floating-point zero (a linear circuit does not mix). The one scale caveat: the conversion matrix is solved densely (capped for modest circuits) on the brute-force shooting PSS. The same solve is the substrate for pnoise and PXF.

<sup>3</sup> Pnoise is △(working, stationary-source first cut). [Enhancement-124](#) adds a `.pnoise` command that folds every device’s noise through the adjoint of the conversion matrix ( $H^T \Psi = e_{\{out, 0\}}$ ): it loads the sideband- $k$  transfer into `CKTrhs/CKTirhs` and calls the existing device noise routines (`NevalSrc`, `OSDI load_noise`) once per sideband, accumulating  $\Sigma_k S \cdot |\Delta\Psi_k|^2$  — so it reuses every device noise model and needs no per-device code. Verified on the RC: because the circuit is linear the conversion matrix is block-diagonal, only sideband 0 contributes, and pnoise reduces exactly to `.noise` ( $4kTR/(1+(2\pi fRC)^2)$ , matching the `.noise` reference to every printed digit). It stays △because the first cut evaluates each device’s noise PSD at the periodic operating-point sample (a stationary approximation of the

cyclostationary source) and does not yet emit a dedicated phase-noise (jitter) spectrum — both refinements of the same fold.

\*4 PXF is ✓ (complete). [Enhancement-125](#) adds a .pxf command — the adjoint of PAC. It solves  $H^T \Psi = e_{\{out, 0\}}$  per frequency and dots each sideband block of  $\Psi$  with the netlist AC-source pattern B0 to get the input→output transfer at each sideband ( $x_f, x_{f\_usb<k>}, x_{f\_lsb<k>}$ ). By the identity  $(H^{-1}B)_{out} = (H^{-T}e_{out})^T B$  the sideband-0 transfer is bit-identical to the PAC response at the output — a reciprocity cross-check that pins the adjoint solve — verified on the RC to equal the analytic low-pass transfer, with the conversion sidebands at floating-point zero. The same dense-solve scale caveat as PAC applies. This completes the PSS → PAC → Pnoise → PXF periodic small-signal suite.

## Performance & scale

| Category    | Feature                                            | ngspice | Spectre |
|-------------|----------------------------------------------------|---------|---------|
| Performance | Multithreaded device-model evaluation (OpenMP)     | ✓       | ✓       |
| Performance | Element bypass / latency exploitation              | ⚠       | ✓       |
| Performance | Fast-SPICE hierarchical / isomorphic engine        | ×       | ✓       |
| Performance | Distributed (MPI) / cloud partitioning             | ×       | ✓       |
| Performance | Handles $10^6$ – $10^8$ -node post-layout netlists | ×       | ✓       |

## Statistical / variability / yield

| Category    | Feature                                                       | ngspice | Spectre |
|-------------|---------------------------------------------------------------|---------|---------|
| Statistical | Monte Carlo                                                   | ⚠       | ✓       |
| Statistical | Native process/mismatch modeling + correlations               | ⚠       | ✓       |
| Statistical | Low-discrepancy sampling (Sobol / Latin-hypercube)            | ×       | ✓       |
| Statistical | High-sigma methods (importance sampling, worst-case distance) | ×       | ✓       |
| Statistical | Corner + MC + yield estimation flow                           | ⚠       | ✓       |

Monte Carlo is ⚠ works, but script-driven (`alter + sgauss`, with the deterministic-seed RNG from [Enhancement-10/66](#)) rather than a native statistical block with sampling controls.

## Reliability / aging

| Category    | Feature                                          | ngspice | Spectre |
|-------------|--------------------------------------------------|---------|---------|
| Reliability | Device aging (HCI / NBTI / TDDB)                 | ×       | ✓       |
| Reliability | Stress → degrade → re-simulate (fresh/aged) flow | ×       | ✓       |
| Reliability | Electromigration + IR-drop (EMIR)                | ×       | ✓       |

## Post-layout / parasitics

| Category    | Feature                                | ngspice | Spectre |
|-------------|----------------------------------------|---------|---------|
| Post-layout | Flat parasitic (RC) netlist simulation | ✓       | ✓       |
| Post-layout | RC reduction / model-order reduction   | ×       | ✓       |
| Post-layout | n-port (S/Y/Z) extracted-block import  | ✓       | ✓       |

## Behavioral / mixed-signal / AMS

| Category | Feature                                    | ngspice | Spectre |
|----------|--------------------------------------------|---------|---------|
| Modeling | Verilog-A (analog, LRM Annex C) via OSDI   | ✓       | ✓       |
| Modeling | XSPIICE code models + event-driven digital | ✓       | ✓       |

| Category | Feature                                    | ngspice | Spectre |
|----------|--------------------------------------------|---------|---------|
| Modeling | Verilog-AMS mixed-signal (connect modules) | ×       | ✓       |
| Modeling | Real-number modeling (wreal / RNM)         | ×       | ✓       |
| Modeling | VHDL-AMS                                   | ×       | ✓       |

## Interfaces & infrastructure

| Category       | Feature                           | ngspice | Spectre |
|----------------|-----------------------------------|---------|---------|
| Interface      | Shared-library / programmatic API | ✓       | ✓       |
| Interface      | Python bindings                   | ✓       | ✓       |
| Infrastructure | Built-in optimizer                | ×       | ✓       |
| Infrastructure | Checkpoint / restart of long runs | ⚠       | ✓       |

## Where to invest (given the Verilog-A/OSDI side is done)

The realistic, winnable identity for this project is the open-source, OSDI/Verilog-A-native simulator with a real RF-noise and statistical story — not out-engineering 25 years of commercial fast-SPICE/parallel work. Ranked by leverage on the existing strength  $\times$  differentiation  $\times$  tractability:

1. RF periodic small-signal suite — PAC  $\rightarrow$  Pnoise  $\rightarrow$  PXF, on the hardened PSS. The PSS foundation is now shipped and verified under both linear solvers ([Enhancement-117](#), [Enhancement-118](#)), and [Enhancement-119](#) retains the periodic operating point (voltages + device states per sample) and [Enhancement-120](#) turns it into the periodic Jacobian harmonics  $G_k$ ,  $C_k$ , and [Enhancement-121](#) assembles those into the  $(2M+1)N$  harmonic conversion matrix and solves it, [Enhancement-122](#) wraps that in a working `.pac` command, and [Enhancement-123](#) finishes it with a netlist-referenced source stimulus and multi-sideband conversion-gain output — a complete periodic-AC analysis, verified against the exact linear transfer and driving-point responses, [Enhancement-124](#) adds `.pnoise` — folding every device's noise through the conversion-matrix adjoint  $H^T$  (reusing the existing device noise routines), verified to reduce exactly to `.noise` in the linear limit — and [Enhancement-125](#) adds `.pxf`, the adjoint transfer function, whose sideband-0 result is bit-identical to the PAC response by reciprocity. The PSS  $\rightarrow$  PAC  $\rightarrow$  Pnoise  $\rightarrow$  PXF periodic small-signal suite is now complete — genuinely novel in open source. The remaining RF work is the refinements (cyclostationary/phase noise, a from-scratch Harmonic Balance engine, quasi-periodic multi-tone) rather than the core analyses.
2. Convergence robustness — coordinated accuracy presets (`errpreset`) landed in [Enhancement-110](#); the remaining piece is pseudo-transient / dynamic-gmin homotopy. Unglamorous but makes every analysis usable on real circuits; self-contained in the ngspice core.
3. High-sigma statistical sampling — high industrial value (yield / SRAM), moderate difficulty, leans on the deterministic-seed RNG already in place.

Explicitly lower priority: fast-SPICE parallelism and aging/EMIR — enormous efforts that don't leverage what makes this project distinctive.